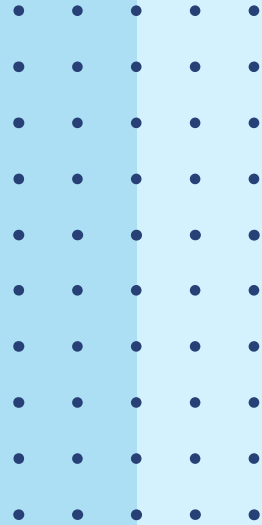


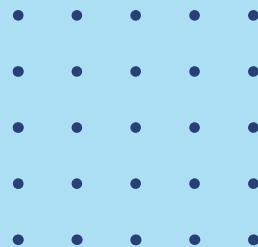
Paging in xv6

Tutorial 3



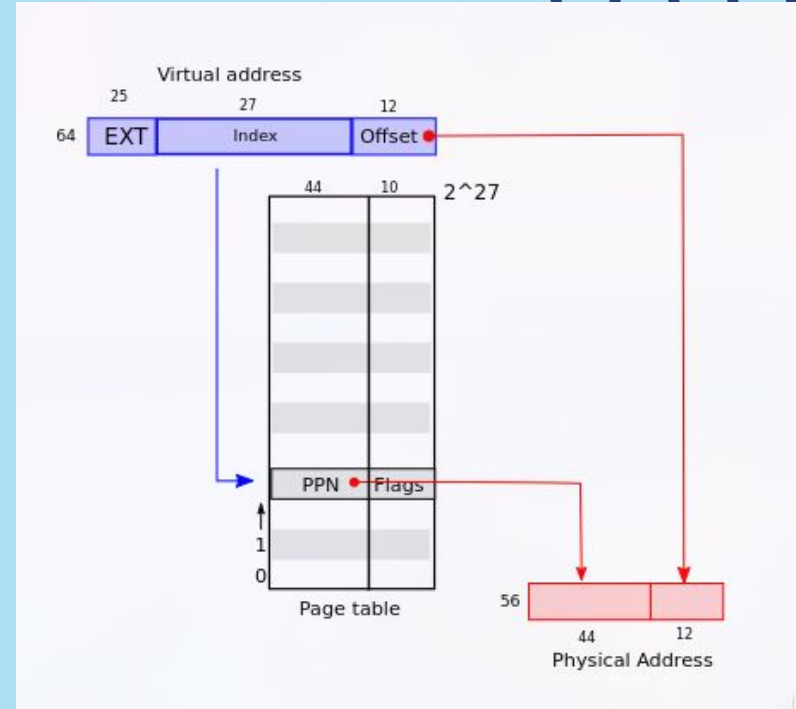
What is paging?

- Chop up the memory space into fixed-size chunks
- This is known as paging
- Each fixed size unit is known as page
- Physical memory -> array of fixed size slots called page frames
- xv6 uses 4 KiB pages



Why page tables?

- Each process has its own page table (hence, they provide process isolation)
- VA map to physical memory
- Permissions control how memory can be accessed

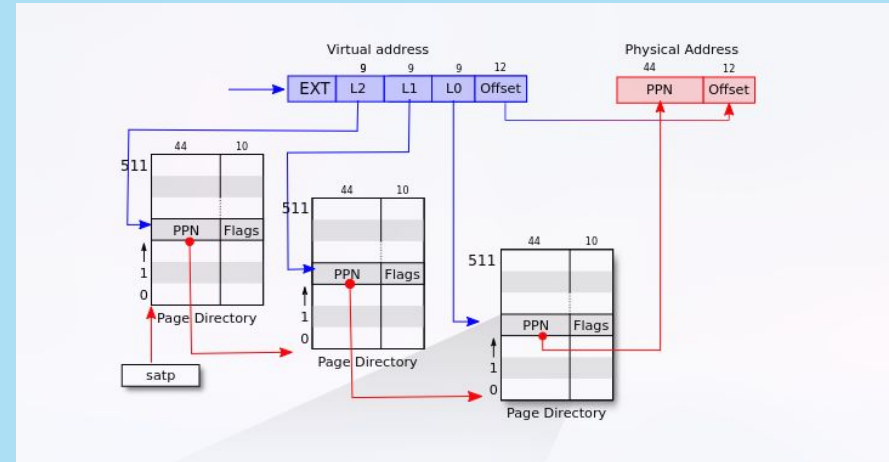


RISC-V Sv39

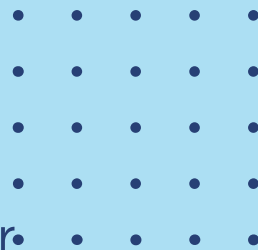
- 39 bit virtual address
- 3 levels of page tables
- 4 KiB pages
- 12-bit offset

$$VA = VPN[2] \mid VPN[1] \mid VPN[0] \mid \text{Offset}$$

What is SATP?



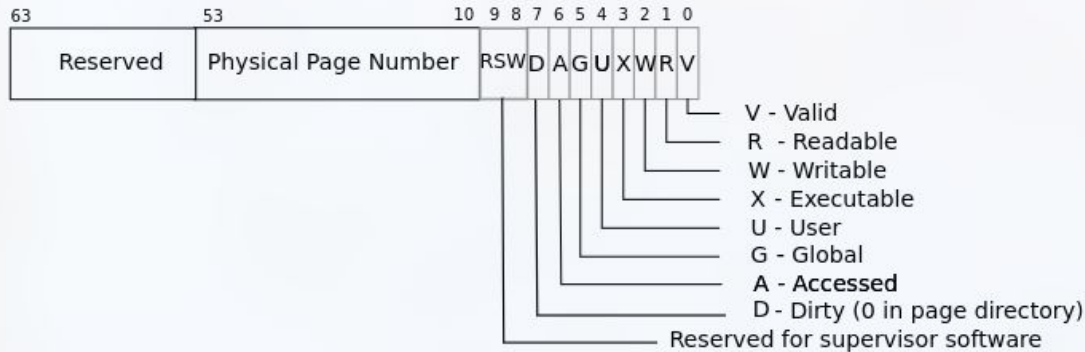
Address Translation



- satp (supervisor address translation and protection) register.
- It points to the root page table
- Hardware walks three levels of page tables
- Each VPN indexes the next level table
- Final PTE contains the physical page number
- Page offset is unchanged during translation
- TLB caches frequently used translations

PTE

- Contains physical page number and access control flags
- V - Valid mapping
- R - Read
- W - Write
- X - Execute
- U - User access mode
- PTE defines both the where the page maps to and how it can be accessed



Global memory mapping

RISC-V Linux Kernel SV39

Start addr	Offset	End addr	Size	VM area description
0000000000000000	0	0000003fffffffff	256 GB	user-space virtual memory, different per mm
0000004000000000	+256 GB	ffffffbfffffffff	~16M TB	... huge, almost 64 bits wide hole of non-canonical virtual memory addresses up to the -256 GB starting offset of kernel mappings.
				Kernel-space virtual memory, shared between all processes:
ffffffc6fea00000	-228 GB	ffffffc6feffffff	6 MB	fixmap
ffffffc6ff000000	-228 GB	ffffffc6ffffffff	16 MB	PCI io
ffffffc700000000	-228 GB	ffffffc7ffffffff	4 GB	vmemmap
ffffffc800000000	-224 GB	ffffffd7ffffffff	64 GB	vmalloc/ioremap space
ffffffd800000000	-160 GB	fffffffeffffffff	124 GB	direct mapping of all physical memory
ffffff7000000000	-36 GB	fffffffeffffffff	32 GB	kasan
ffffff0000000000	-4 GB	ffffff7fffffffff	2 GB	modules, BPF
ffffff8000000000	-2 GB	ffffffffffffffff	2 GB	kernel

Kernel address space

- Single page table describing a kernel's address space
- Different types of mapping
 - Direct mapping - maps all physical RAM and device registers
 - Trampoline - mapped at the top of the virtual address space; this is used for user page tables as well
 - Kernel stack pages - kernel stack mapped at a high kernel VA. xv6 leaves an unmapped guard page below it

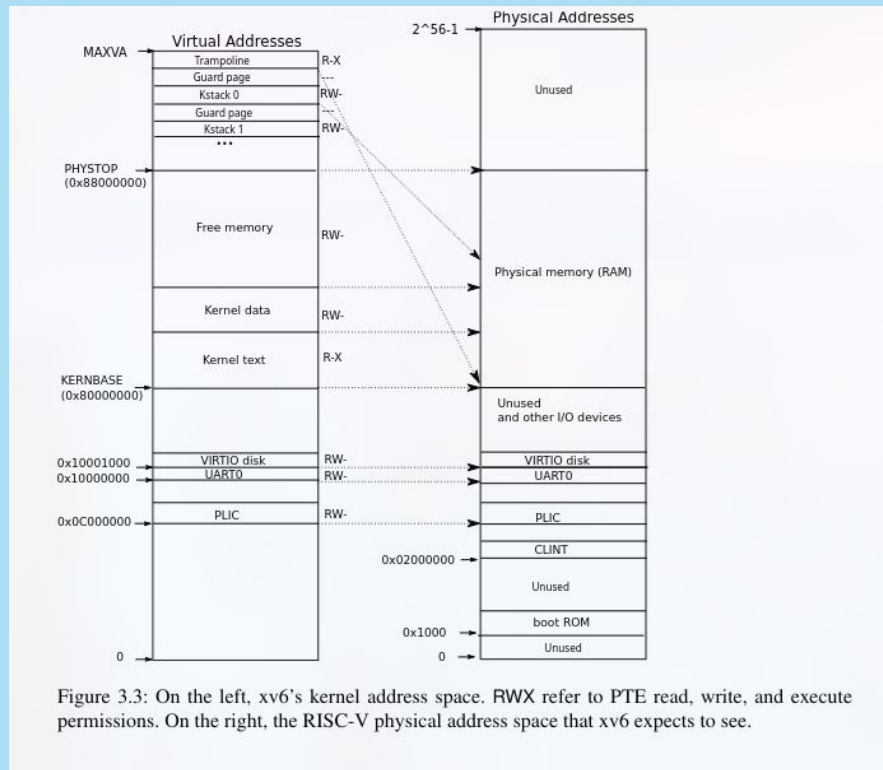
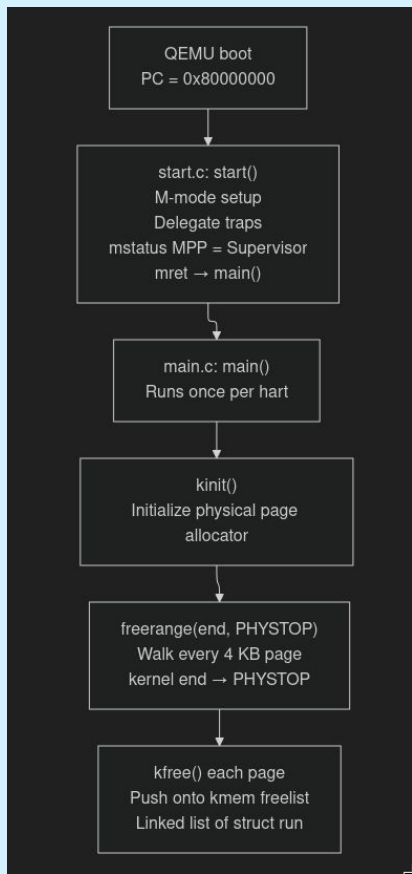
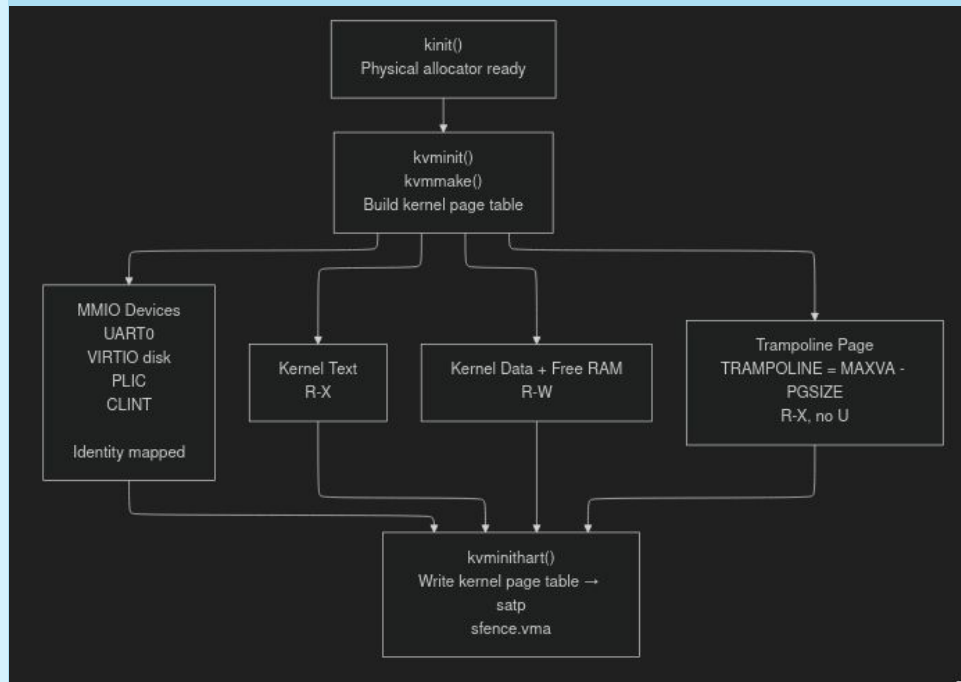


Figure 3.3: On the left, xv6's kernel address space. RWX refer to PTE read, write, and execute permissions. On the right, the RISC-V physical address space that xv6 expects to see.



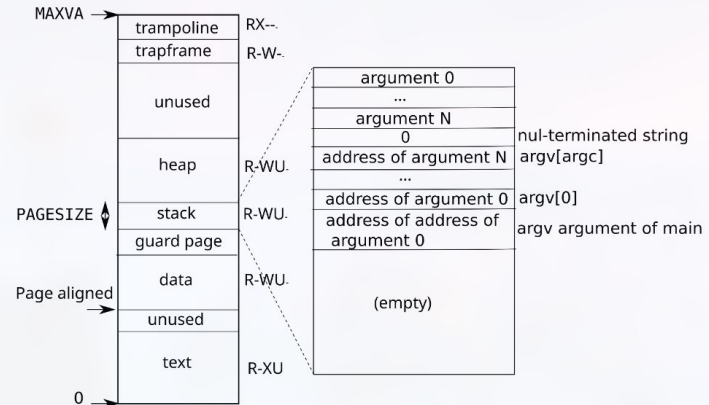
Allocating physical memory

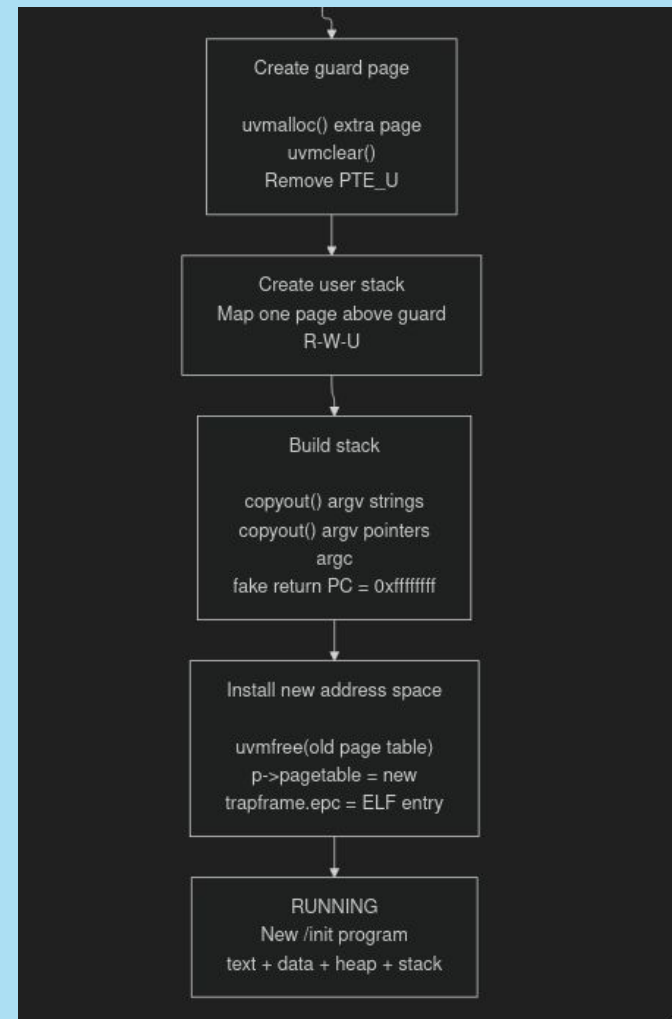
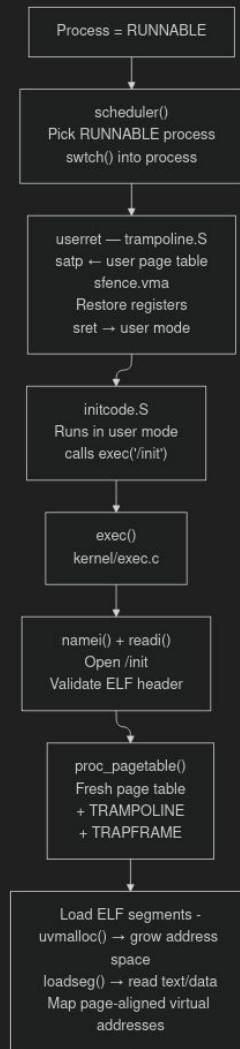


Creating an address space

Process Address Space

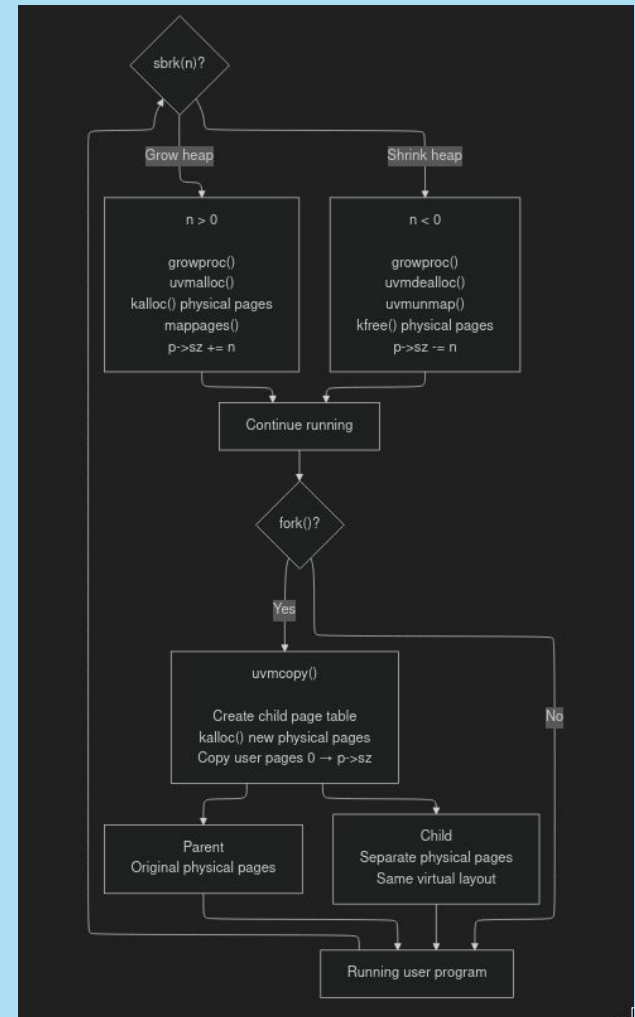
- Consists of pages that contain the text, data and a page for stack and heap
 - PTE_R, PTE_W, PTE_U are the flags used for the mapping heap, stack and data. text does not have PTE_W perms, instead has PTE_X -> to prevent a program from accidentally modifying its own code
 - User programs are mapped without specific permissions, to prevent programs from going rouge
 - If a user stack overflows and the process tries to use an address below the stack, the hardware will generate a page-fault exception (guard page is inaccessible to a program running in user mode)
-
- Each process has private user memory
 - Each process sees its memory as having contiguous virtual addresses
 - Trampoline: Mapped at the same location for both user and kernel (shared entity between the user and the kernel)
 - Trapframe: Stores the program's memory before going into kernel mode and restores it when its back





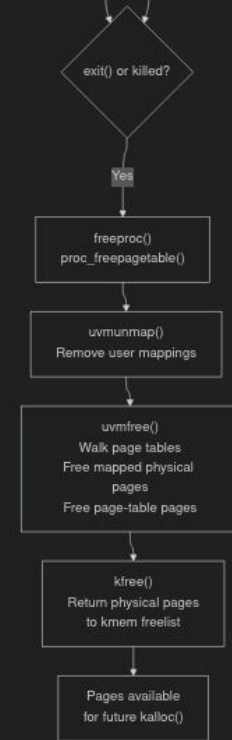
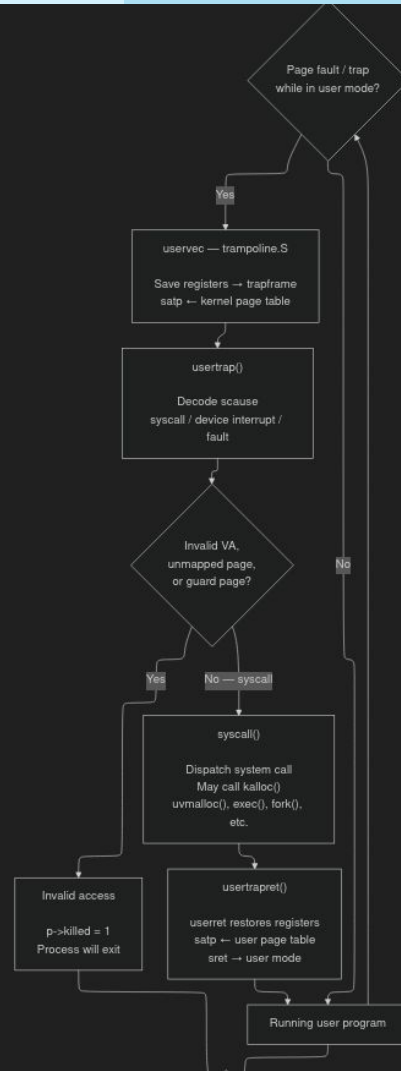
Growing user memory

- `sbrk(n)` changes the process's size
- Positive `n` -> grow the heap
- `uvmmalloc()` allocates physical pages
- `kalloc()` obtains the physical memory
- `mappages()` creates the virtual mappings
- Negative `n` -> release pages

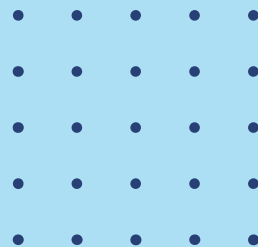


Trap Handling

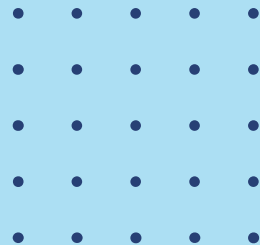
- User program triggers a trap
- uservec saves registers in trapframe
- Kernel page table becomes active
- Cause (syscall, device interrupt, page fault) is determined by usertrap()



Page faults and Lazy allocation



- Page faults happen when a virtual address that has no mapping in physical memory is used
- `sbrk()` is the system call for a process to shrink or grow its memory
- When a process asks for memory with the flag `SBRK_LAZY`, the kernel notes the increase in size, but no physical memory will be allocated to the process and no PTEs will be made
- Physical memory is mapped to a VA only when a page fault happens on these addresses
- If the flag `SBRK_EAGER` is passed, then lazy allocation will not happen and memory will be assigned for a process as soon as it asks for it



THANK YOU